

Table of Contents

1. Introduction	(Page No:3 - 4)
1.1 Purpose	
1.2 Intended Customers/Users	
1.3 Product Scope	
1.4 Outline of the document	
2. Overall Description	(Page No:4 - 5)
2.1 Product Perspective	
2.2 Product Functions	
2.3 User Classes	
2.4 Design and Implementation Constraints	
3. Diagrams	(Page No: 5 - 20)
3.1 UML Diagrams	(Page No: 5 - 14)
3.2 ER Diagrams	
3.3 DFD Diagrams	(Page No: 14 - 20)
3.4 Assumptions and Dependencies	
4. External Interface Requirements/Screenshots	
4.1 User Interfaces	
4.2 Hardware Interfaces	
4.3 Software Interfaces	
4.4 Communication Interfaces	
5. Other Non-Functional Requirements	
5.1 Performance Requirements	
5.2 Safety Requirements	
5.3 Security Requirements	
5.4 Software Quality Attributes	
5.5 Business Policies/Rules (if any)	
6. Testing	(Page No: 21 - 26)
6.1 Block-Box Testing	
6.2 White-Box Testing	
7. Design Architecture of System	(Page No: 26 - 28)
8. Function Based Metrics	(Page No: 29 - 31)
9. COCOMO Model	(Page No: 31 - 34)
10. Conclusion	
11. References	
12. Acknowledgements	

1 Introduction

The AI Based Food Analyzer is a software system designed to help users understand the health impact of additives present in packaged food products. Many packaged foods contain chemical additives whose effects are not easily understood by common consumers. This system uses artificial intelligence and a predefined additive database to analyze these additives and classify them into different harm levels. The system aims to promote healthier food choices by providing clear, color-coded results that indicate whether a food additive is safe, moderately harmful, or harmful.

1.1 Purpose

The purpose of this Software Requirement Specification (SRS) document is to describe the functional and non-functional requirements of the AI Based Food Analyzer system.

This document serves as a reference for:

- Developers who will build the system
- Testers who will verify the system
- Faculty and evaluators who will assess the project

It clearly explains what the system will do, how it will behave, and the constraints under which it will operate.

1.2 Intended Customers

The AI Based Food Analyzer is intended for the following users:

1. **General Consumers**
People who want to check whether packaged food additives are safe to consume.
2. **Students**
Students studying nutrition, food technology, or health sciences.
3. **Health-Conscious Individuals**
Users who actively monitor their diet and avoid harmful food ingredients.
4. **Nutritionists and Dieticians**
Professionals who want quick reference information on food additives.

1.3 Product Scope

The scope of the AI Based Food Analyzer includes:

- Allowing users to enter food additives manually
 - Analyzing additives using an AI-based rule system
 - Categorizing additives into:
 - Green (Safe)
 - Yellow (Moderate Risk)
 - Red (Harmful)
 - Displaying clear and easy-to-understand results
 - Providing basic information about each additive
- The system does **not** diagnose diseases or replace medical advice.

1.4 Outline of the Document

This document is organized as follows:

- **Section 1** describes the introduction, purpose, users, and scope.
- **Section 2** explains the overall system description and features.
- **Section 3** covers diagrams and assumptions.
- **Section 4** describes external interfaces.
- **Section 5** lists non-functional requirements.
- **Section 6** explains testing strategies.
- **Section 7–9** include conclusion, references, and acknowledgements.

2. Overall Description

This section provides a high-level overview of the AI Based Food Analyzer system, including how it works, who uses it, and its limitations.

2.1 Product Perspective

The AI Based Food Analyzer is a **web-based application** that works as a standalone system. Users access the system using a web browser on a computer or mobile device.

The system interacts with:

- A food additive database
- An AI-based classification logic
- A user interface for input and output

In future, it can be integrated with barcode scanners or food-label recognition systems

2.2 Product Functions

The main functions of the system are:

- User registration and login
- Manual input of food additives
- AI-based analysis of additives
- Classification of additives into harm levels
- Color-coded result display
- Display of additive descriptions and effects
- Admin management of additive database

2.3 User Classes

The system supports the following user classes:

1. General User

- Enter food additives
- View analysis results
- Understand harm levels

2. Administrator

- Manage additive database
- Update harm classifications

2.4 Design and Implementation Constraints

The system has the following constraints:

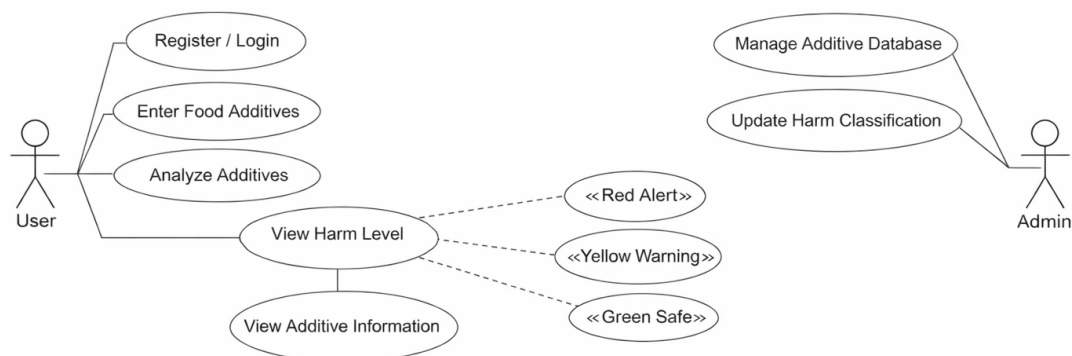
- Requires a stable internet connection
- Depends on the accuracy of the additive database
- Requires modern web browsers
- AI accuracy is limited to available data
- Data privacy rules must be followed

3. Diagrams

This section explains the diagrams used to design the system.

3.1 UML Diagram

UML diagrams describe the system structure and interactions, including:



Actors:

1. User:

The User is the main actor who interacts with the system to check and evaluate food additives present in packaged food products.

2. Admin:

The Admin manages and maintains the system by updating additive information and ensuring that the database and rules used by the system remain accurate.

User Use Cases:

1. Register / Login:

Before accessing the system features, the user needs to create an account or log in using existing credentials. This step ensures that only authorized users can use the application.

2. Enter Food Additives:

The user inputs the additives listed on the packaged food product into the system so they can be evaluated.

3. Analyze Additives:

The system analyzes the entered additives using its built-in logic and stored database information to determine their potential effects.

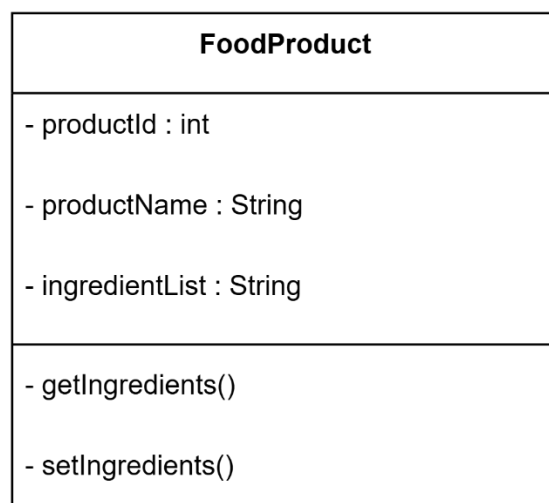
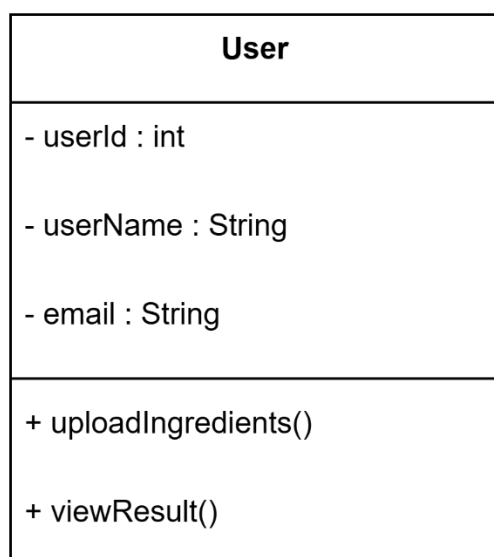
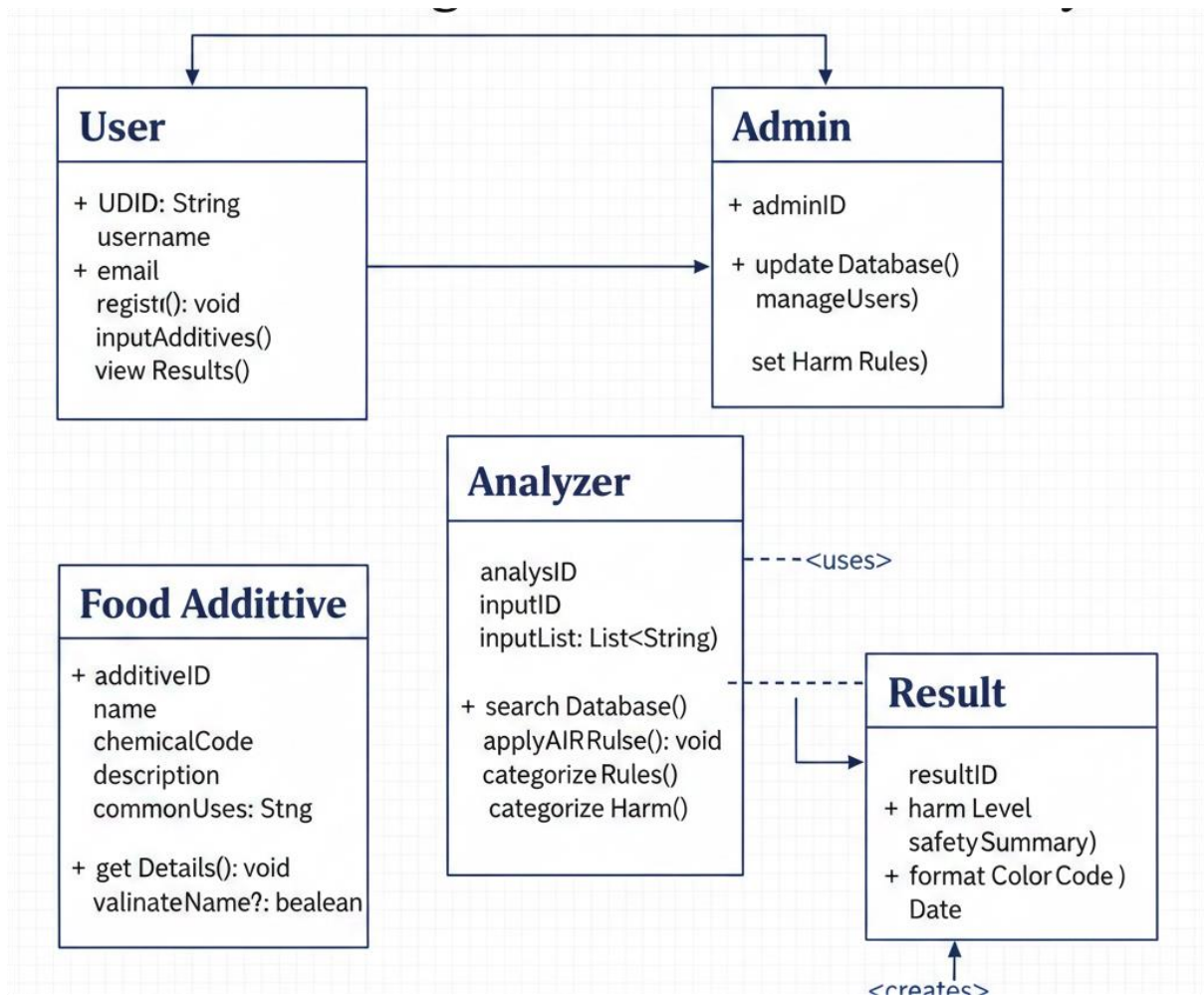
4. View Harm Level:

After the analysis is completed, the system displays the safety level of the additives, indicating whether they are safe, moderately risky, or harmful.

5. View Additive Information:

The user can access additional details about each additive, including its purpose, possible health impacts, and safety classification.

3.2 UML Class Diagram



IngredientAnalyzer	HealthEvaluator
- redIngredients : List - orangeIngredients : List - greenIngredients : List	- greenScore : int - orangeScore : int - redScore : int
+ analyzeIngredients() + classifyIngredient()	+ calculateScore() + determineHealthLevel()

ResultDisplay
- healthStatus : String - warningMessage : String
+ showResult() + showWarnings()

1. Core Classes and Their Attributes

• User Class:

The User class represents a person who interacts with the system to analyze food additives. It stores basic information such as UDID (Unique Identifier), username, and email.

Methods:

- register() – Allows the user to create an account.
- inputAdditives() – Enables the user to enter the additives found in a food product.
- viewResults() – Displays the analysis results generated by the system.

• Admin Class:

The Admin class is an extended version of the User with additional system privileges and control functions.

Methods:

- updateDatabase() – Updates additive information stored in the system.
- manageUsers() – Manages user accounts and system access.
- setHarmRules() – Defines or modifies the rules used by the AI to classify additives based on their safety level.

- **Food Additive Class:**

The Food Additive class represents the data structure used to store information about food ingredients.

Attributes:

- chemicalCode – The identification code of the additive (e.g., E124).
- description – Explanation or details about the additive.
- commonUses – Information about where the additive is commonly used.

Method:

- getDetails() – Retrieves additive information required for analysis.

- **Analyzer Class:**

The Analyzer class acts as the core processing unit of the system. It handles the analysis logic and determines the safety level of additives.

Methods:

- searchDatabase() – Looks for additive data in the system database.
- applyAIRules() – Uses predefined AI rules to classify additives based on their risk level.

- **Result Class:**

The Result class represents the output generated after the additive analysis is completed.

Attributes:

- harmLevel – Indicates the safety category of the additive.
- date – Records the date when the analysis was performed.

Method:

- formatColorCode() – Displays the result using color indicators such as Red, Yellow, or Green.

2. Relationships Between Classes:

- User → Admin

This relationship represents inheritance. The Admin class is derived from the User class, meaning an Admin has all the properties of a User along with additional capabilities.

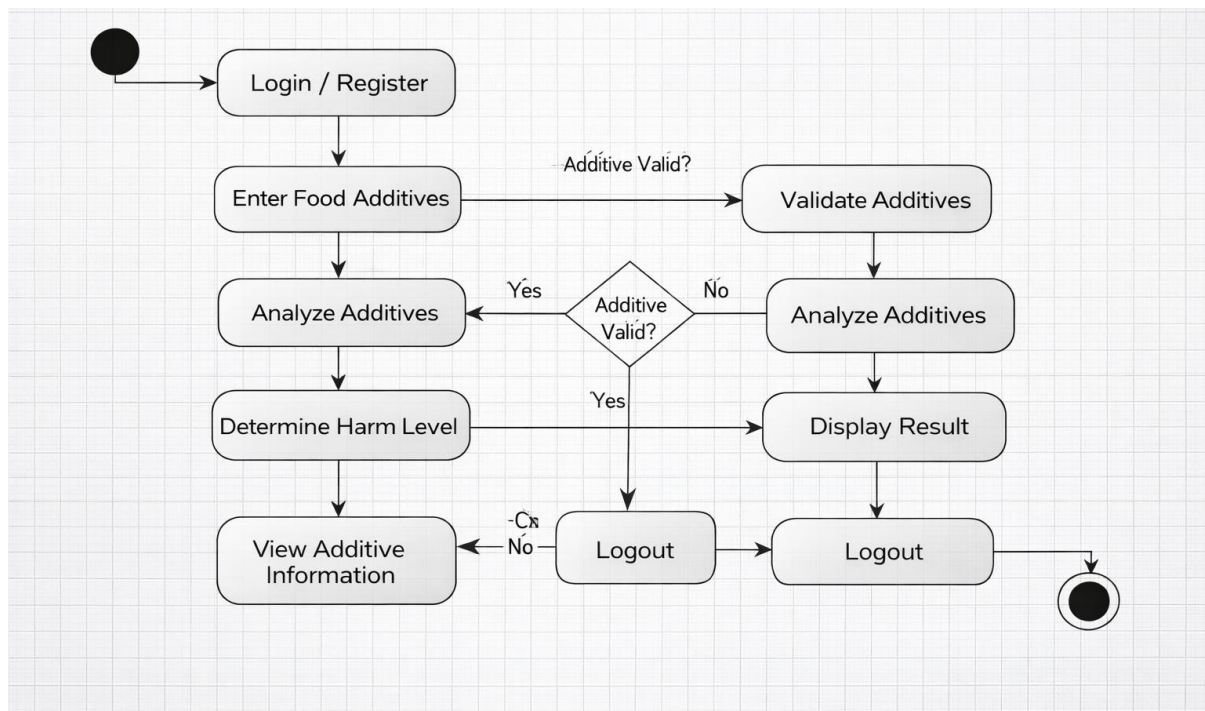
- Analyzer → Food Additive

This relationship indicates a dependency (labeled <uses>). The Analyzer relies on information from the Food Additive class to perform the analysis.

- Analyzer → Result

This represents an association (labeled <creates>). After processing the additives, the Analyzer generates a Result object that contains the final analysis outcome.

3.3 UML Chart Diagram:



Start State:

The process begins from the initial state, represented by a filled circle. From this point, the system starts the workflow and moves to the login stage.

Login / Register:

In this state, the user either creates a new account or logs into the system. Once the authentication process is successful, the system allows the user to proceed to the next stage.

Enter Food Additives:

After logging in, the user provides the list of food additives found in the packaged product. These additives are entered into the system for analysis.

Validate Additives:

The system verifies whether the entered additives exist in the database and checks if the input format is correct. This stage ensures that only valid additives are processed.

Decision: Additive Valid

At this decision point, the system determines whether the additive information is valid. If the additive is valid, the system continues with the analysis process. Otherwise, the user may need to re-enter the additive information.

Analyze Additives:

In this state, the system evaluates the additives using predefined rules or stored data. The analyzer processes the information to identify possible health risks.

Determine Harm Level:

Based on the analysis results, the system categorizes the additive into different safety levels such as safe, moderate risk, or harmful.

Display Result:

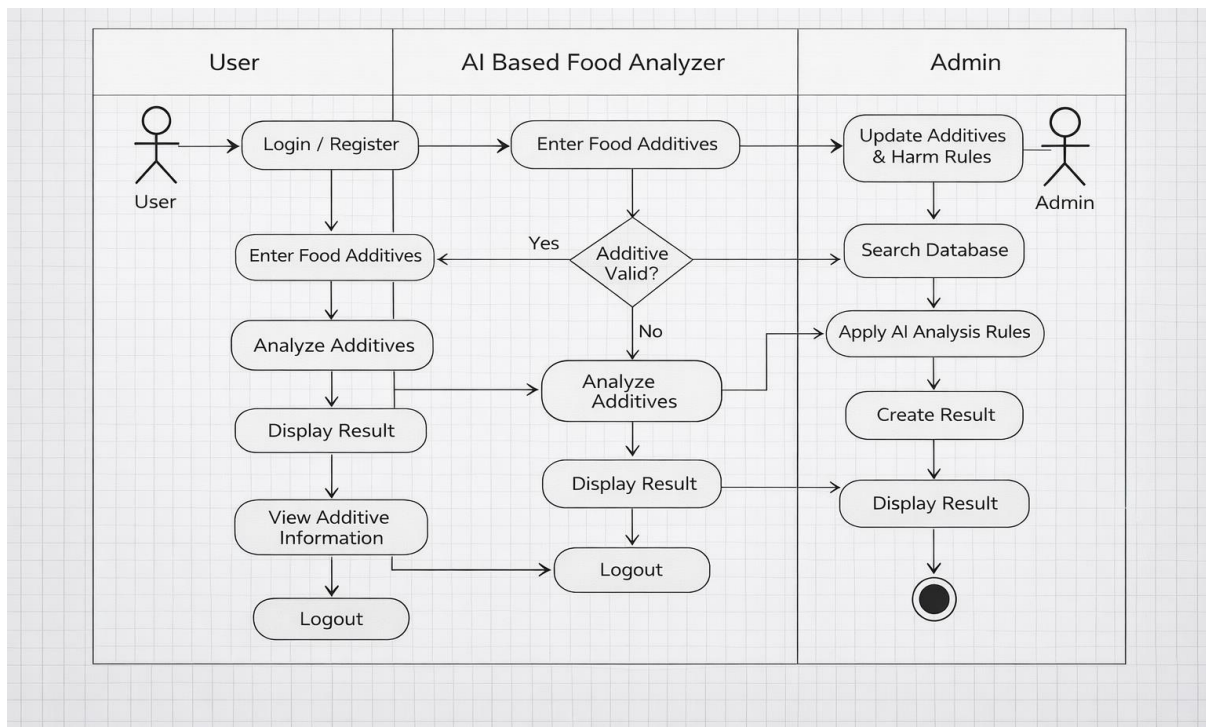
The system then presents the analysis outcome to the user, showing the safety level of the additives.

View Additive Information:

Users can also access additional details about the additives, including their effects and risk levels.

Logout and End State:

Finally, the user logs out of the system. The process ends at the final state, which indicates the completion of the interaction.

3.4 UML Activity Diagram:

Start of the Process:

The process begins when the user accesses the system and logs in or registers. After successful authentication, the user is allowed to proceed with the analysis process.

Entering Food Additives:

Once logged in, the user enters the food additives listed on the packaged food product. These additives are submitted to the system for verification and processing.

Validation of Additives:

The system checks whether the entered additives are valid and available in the database. A decision point determines if the additive information is correct.

- If the additive is valid, the system proceeds with the analysis.
- If the additive is invalid, the user may need to enter the additive information again.

Additive Analysis;

After validation, the system performs the analysis of additives using the stored database and predefined rules. The analyzer evaluates the safety level of each additive.

Result Generation:

Based on the analysis, the system generates the result that indicates the harm level of the additives. The result is then displayed to the user.

Viewing Additional Information:

The user can also view more details about the additives, including their properties and potential health effects.

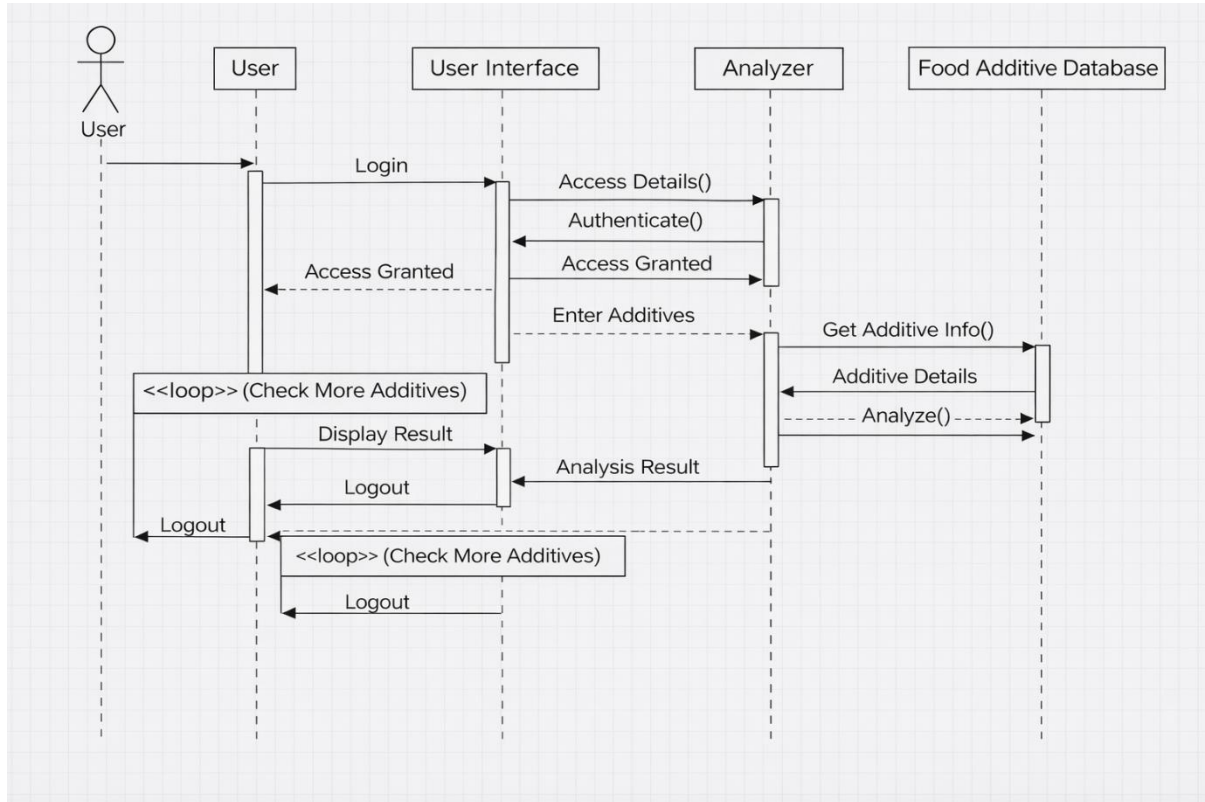
Admin Activities:

At the same time, the admin manages the system database. The admin can update additive information and modify the rules used for harm classification. These updates help the system maintain accurate and up-to-date analysis results.

End of the Process:

Finally, the user logs out of the system, which marks the end of the activity flow.

3.4 UML Sequence Diagram:



Step 1: Login

The process begins when the user attempts to log into the system through the User Interface. The interface sends the user's login information to the Analyzer, which checks whether the credentials are correct. If the authentication is successful, the Analyzer returns an "Access Granted" response. The User Interface then informs the user that they have successfully logged in and can proceed to use the system.

Step 2: Enter Additives

After logging in, the user enters the food additives listed on the packaged product. These additives are submitted through the User Interface, which forwards the information to the Analyzer so that the system can begin processing them.

Step 3: Retrieve Additive Information

The Analyzer needs detailed information about the additives before performing any analysis. To do this, it sends a request to the Food Additive Database. The database responds by providing the stored information related to the entered additives. This data is then used by the system for further evaluation.

Step 4: Analysis Process

Once the required information is collected, the Analyzer processes the additives using its predefined rules and logic. Based on this evaluation, the system determines the safety level of each additive, classifying them as Green (Safe), Yellow (Moderate Risk), or Red (Harmful).

Step 5: Displaying the Result

After the analysis is completed, the Analyzer sends the result back to the User Interface. The interface then displays the final output to the user so they can clearly see the safety level of the additives present in the product.

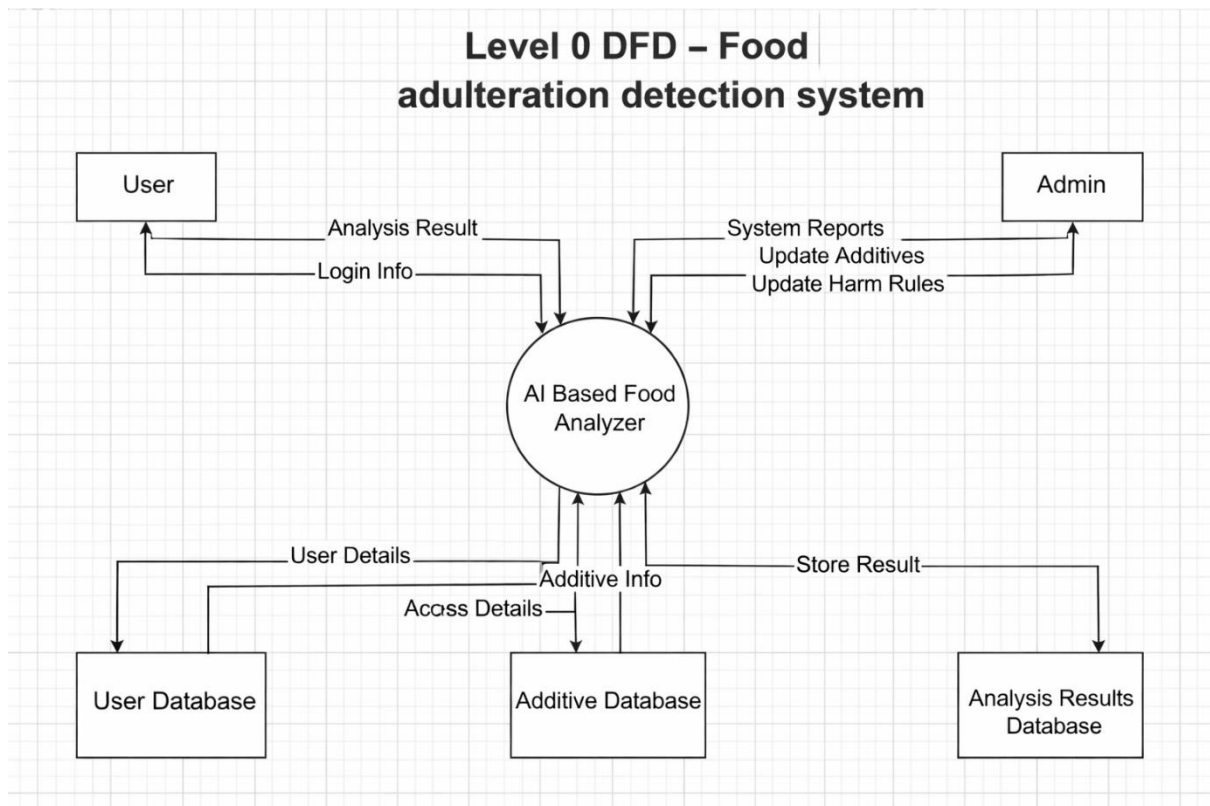
Loop: Checking More Additives

The loop section in the diagram indicates that the user can repeat the analysis process multiple times. If the user wants to check additional additives, the system allows them to enter new inputs and repeat the analysis steps again.

Step 6: Logout

Finally, when the user finishes using the system, they can choose the logout option. This ends the current session and returns the system to its initial state, ready for another user.

3.5 DFD Diagram:



1. Central Process:

AI Based Food Analyzer

The AI Based Food Analyzer acts as the main processing unit of the system. It controls the overall functioning and manages the flow of information between users, administrators, and the databases.

The system performs several important tasks such as:

- Receiving input from the user
- Examining the entered food additives
- Retrieving required information from the databases
- Generating the final result that shows the harm level of the additives

All interactions and data transfers within the system pass through this central process.

2. External Entities

User

The User is the person who uses the application to check the safety of food additives. The user communicates with the system by:

- Sending login information to access the system
- Entering food additive details for analysis
- Viewing the results generated by the system

Admin

The Admin is responsible for maintaining and controlling the system. The administrator performs management tasks such as:

- Updating the list of additives in the database
- Modifying the rules used to determine the harm level
- Reviewing system reports to monitor system activity

3. Data Stores (Databases)

User Database

The User Database keeps records related to users of the system.

It stores information such as:

- User profile details
- Login credentials

Data Flow:

- The system sends user information to the database for verification.
- The database then returns the required access details to the system.

Additive Database

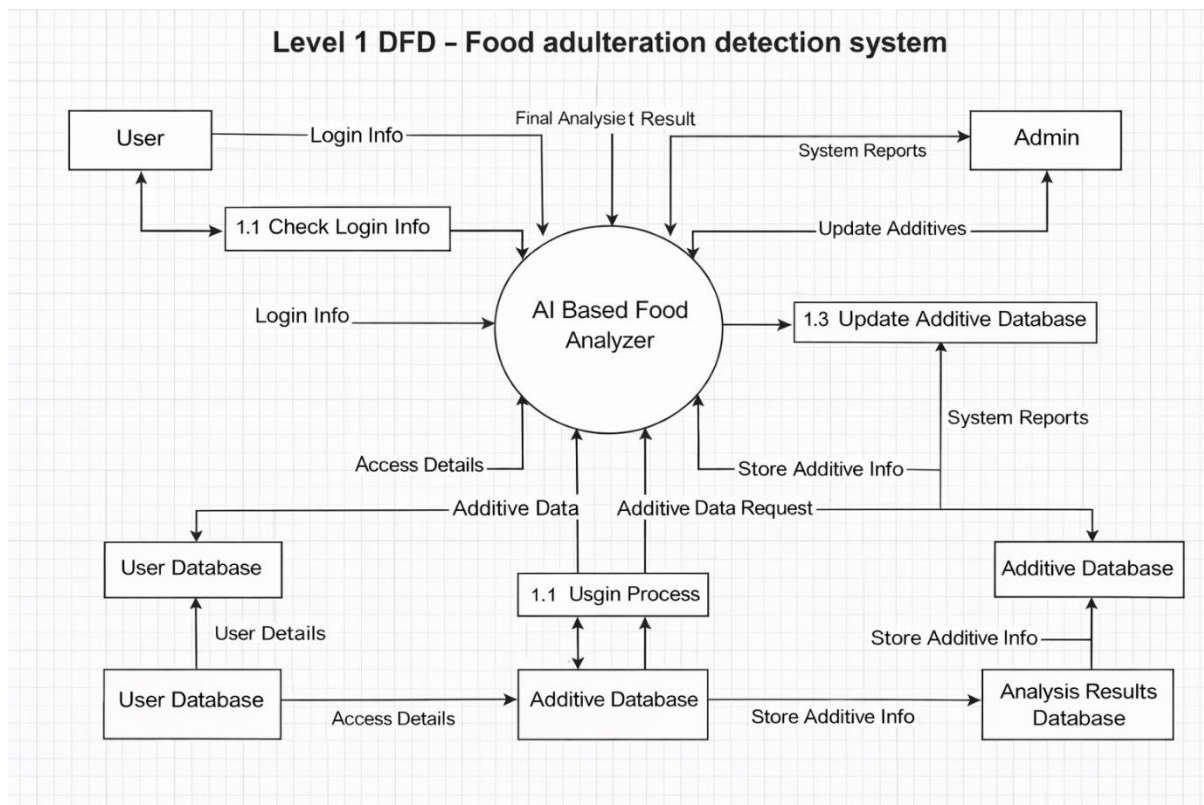
The Additive Database contains information about different food additives used in packaged products.

It stores:

- Names of additives
- Safety or harm-level classification

Data Flow:

- The system sends a request to obtain additive information.
- The database responds by providing the necessary additive data used for analysis.



3.6 Level 1 DFD Explanation:

Food Adulteration Detection System:

The Level 1 DFD provides a detailed breakdown of the main process shown in Level 0. The central system, AI Based Food Analyzer, is divided into smaller sub-processes to clearly show how data flows within the system.

1. Main Process Decomposition:

The central system is divided into the following sub-processes:

1.1 Login Verification:

This process checks whether the user's login details are valid.

- The user enters login information.
- The system verifies it using data stored in the User Database.
- If valid, access is granted to the user.

1.2 Additive Analysis Process:

This is the core functionality of the system.

- The user provides food additive details.
- The system retrieves relevant information from the Additive Database.
- The additives are analyzed based on predefined rules.

- The system generates the final result showing whether the additive is safe or harmful.

1.3 Update Additive Database:

This process is handled by the admin.

- The admin updates additive information and harm classification rules.
- The updated data is stored in the Additive Database.
- These updates help improve the accuracy of analysis.

2. External Entities:

User

- Sends login credentials to the system
- Provides additive details for analysis
- Receives the final analysis result

Admin

- Updates additive data and rules
- Receives system reports
- Maintains overall system accuracy

3. Data Stores:

User Database

- Stores user information such as login credentials and profiles
- Provides access details during login verification

Additive Database

- Stores information about food additives
- Provides data required for analysis
- Gets updated by the admin

Analysis Results Database

- Stores the results generated after analysis
- Helps in maintaining history and generating reports

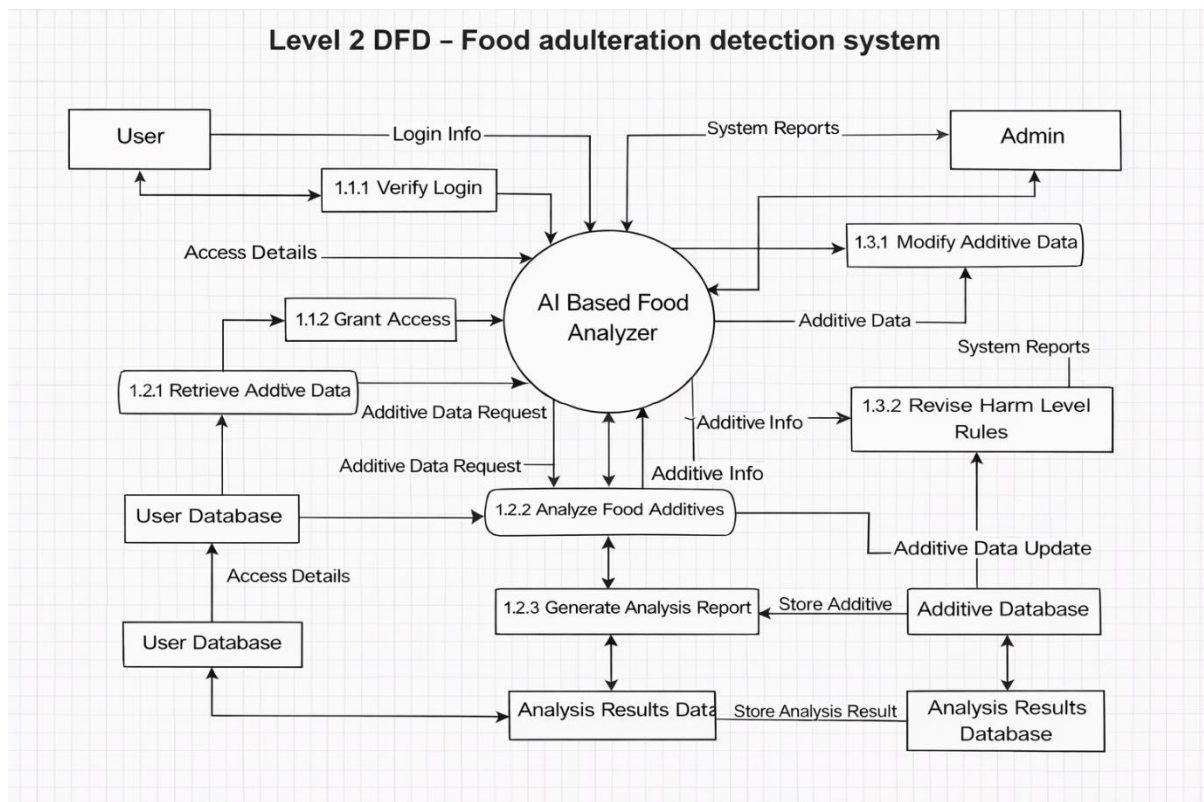
4. Data Flow:

The system works in the following sequence:

1. The user sends login details → system verifies using User Database
2. After login, user enters additive data
3. System requests information from Additive Database
4. Analysis is performed
5. Result is generated and stored in Analysis Results Database
6. Final result is displayed to the user
7. Admin can update additive data and view reports

5. Conclusion:

The Level 1 DFD clearly shows how the system is divided into smaller processes such as login checking, additive analysis, and database updates. It also illustrates how data moves between users, admin, and different databases, making the system workflow easy to understand.



3.7 Level 2 DFD Explanation:

Food Adulteration Detection System:

The Level 2 DFD gives a more detailed view of the system by breaking down the Level 1 processes into smaller steps. It explains how each function works internally and how data moves between users, processes, and databases.

1. User Authentication Process:

1.1.1 Verify Login

- The user enters login credentials.
- The system checks these details with the User Database.
- If the data matches, the user is verified.

1.1.2 Grant Access

- After successful verification, access is provided to the user.
- The system sends confirmation and allows further operations.

2. Additive Data Processing:

1.2.1 Retrieve Additive Data

- The system requests additive-related information from the Additive Database.
- The database returns details such as additive name and classification data.

1.2.2 Analyze Food Additives

- The retrieved data is processed using rules or AI logic.
- The system compares additives with stored information.
- Based on this, it determines whether the additive is safe, moderate, or harmful.

1.2.3 Generate Analysis Report

- After analysis, a result is created.
- The result is prepared in a user-friendly format.
- It shows the safety level of the additive clearly.

3. Result Storage:

- The generated result is stored in the Analysis Results Database.
- This helps maintain history and allows future reference or reporting.

4. Admin Control Processes:

1.3.1 Modify Additive Data

- The admin updates or adds new additive information.
- Changes are saved in the Additive Database.

1.3.2 Revise Harm Level Rules

- The admin can update classification rules.
- These rules are used during analysis to determine safety levels.

5. Data Stores:

User Database

- Stores login credentials and user details
- Used during authentication

Additive Database

- Stores additive-related data
- Provides information for analysis
- Gets updated by admin

Analysis Results Database

- Stores results after processing
- Used for reports and tracking

6. Overall Flow:

1. User logs in → system verifies using User Database
2. User enters additive details
3. System retrieves data from Additive Database
4. Additives are analyzed using rules
5. Result is generated and stored
6. Output is shown to the user
7. Admin updates data and rules when required

7. Conclusion:

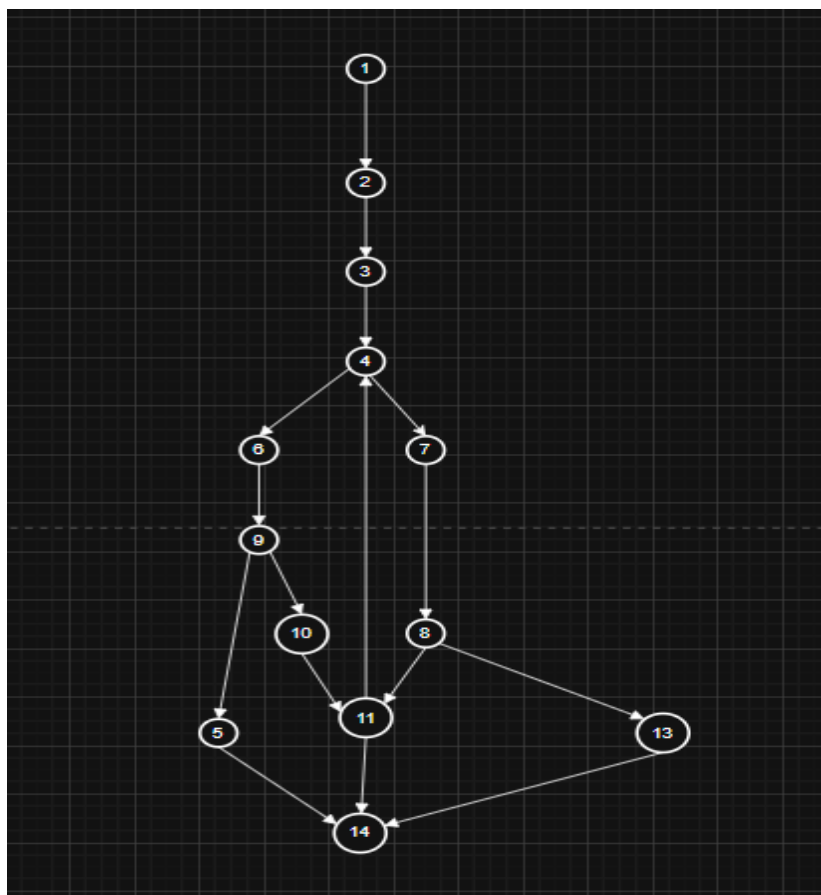
The Level 2 DFD explains the internal working of the system in detail by dividing each process into smaller steps. It clearly shows how authentication, data retrieval, analysis, and result storage are handled, making the system design more understandable.

6.Testing:

Data Flow Graph:

Node Meaning

- 1 Start
- 2 User Login
- 3 Validate Credentials
- 4 Enter Additives
- 5 User Database
- 6 Additive Found Check
- 7 Fetch Additive Info
- 8 Analyze Harm Level
- 9 Decision Node
- 10 Store Result
- 11 Enter More Additives
- 13 Logout
- 14 End
- 15 Final Output (optional)



6.1 Black Box Testing:**Harm Score Validation Boundaries (0 and 100):**

BB No	Test Input (Harm Score)	Expected Output	Technique
BB1	[-1]	Invalid Score	Below 0
BB2	[0]	Green (Safe)	Boundary 0
BB3	[1]	Green (Safe)	Above 0
BB4	[99]	Red (Harmful)	Below 100
BB5	[100]	Red (Harmful)	Boundary 100
BB6	[101]	Invalid Score	Above 100

Safe Boundary at 30:

BB No	Test Input	Expected Output	Technique
BB7	[29]	Green (Safe)	30 – 1
BB8	[30]	Yellow (Moderate Risk)	Boundary 30
BB9	[31]	Yellow (Moderate Risk)	30 + 1

Moderate Risk Boundary at 60:

BB No	Test Input	Expected Output	Technique
BB10	[59]	Yellow (Moderate Risk)	60 – 1
BB11	[60]	Red (Harmful)	Boundary 60
BB12	[61]	Red (Harmful)	60 + 1

High Risk Boundary at 80:

BB No	Test Input	Expected Output	Technique
BB13	[79]	Red (Harmful)	80 – 1
BB14	[80]	Red (Harmful)	Boundary 80
BB15	[81]	Red (Harmful)	80 + 1

Program_1:

```
*food adulteration.py - C:/Users/sanke/OneDrive/Documents/food adulteration.py (3.11.9)*
File Edit Format Run Options Window Help

import unittest

additive_db = {
    "E100": "Green Safe",
    "E101": "Green Safe",
    "E102": "Yellow Warning",
    "E211": "Yellow Warning",
    "E220": "Red Alert",
    "E621": "Red Alert"
}

def analyze_additives(additives):

    results = []

    for additive in additives:
        if additive in additive_db:
            results.append((additive, additive_db[additive]))
        else:
            results.append((additive, "Unknown Additive"))

    return results

if __name__ == "__main__":

    print("AI Based Food Analyzer")
    print("Enter additive codes separated by space (Example: E100 E102 E621):")

    user_input = input()

    additives = user_input.upper().split()

    results = analyze_additives(additives)

    print("\nAnalysis Result:")

    for additive, status in results:
        print(additive, "->", status)

class TestFoodAnalyzer(unittest.TestCase):

    def test_green(self):
        self.assertEqual(analyze_additives(["E100"]), [("E100", "Green Safe")])

    def test_yellow(self):
```

```

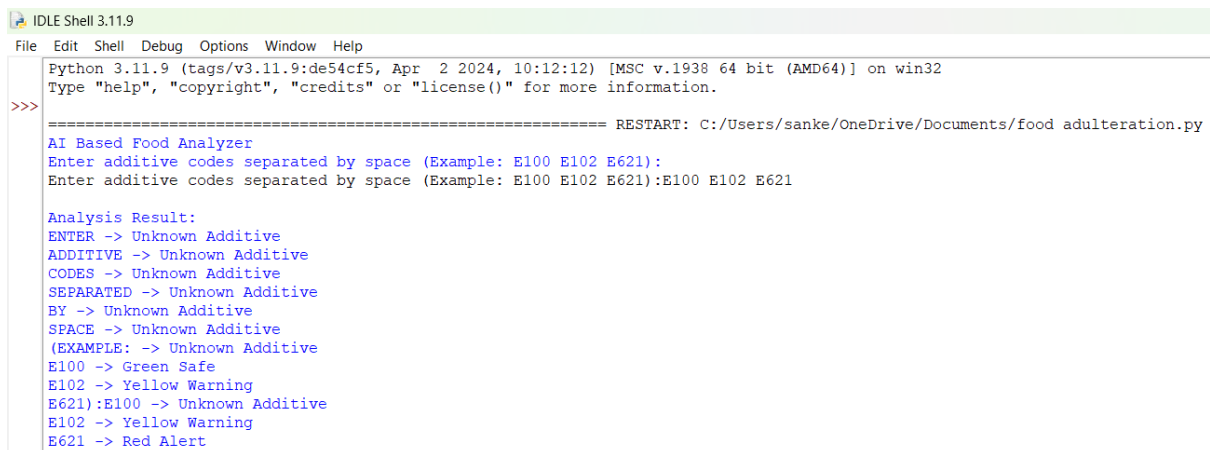
def test_red(self):
    self.assertEqual(analyze_additives(["E621"]), [("E621", "Red Alert")])

def test_unknown(self):
    self.assertEqual(analyze_additives(["E999"]), [("E999", "Unknown Additive")])

def test_multiple(self):
    self.assertEqual(
        analyze_additives(["E100", "E102", "E621"]),
        [
            ("E100", "Green Safe"),
            ("E102", "Yellow Warning"),
            ("E621", "Red Alert")
        ]
    )

```

Output:



```

IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/sanke/OneDrive/Documents/food adulteration.py
AI Based Food Analyzer
Enter additive codes separated by space (Example: E100 E102 E621):
Enter additive codes separated by space (Example: E100 E102 E621):E100 E102 E621

Analysis Result:
ENTER -> Unknown Additive
ADDITIVE -> Unknown Additive
CODES -> Unknown Additive
SEPARATED -> Unknown Additive
BY -> Unknown Additive
SPACE -> Unknown Additive
(EXAMPLE: -> Unknown Additive
E100 -> Green Safe
E102 -> Yellow Warning
E621):E100 -> Unknown Additive
E102 -> Yellow Warning
E621 -> Red Alert

```


Program_2:

```
food adulteration.py - C:/Users/sanke/OneDrive/Documents/food adulteration.py (3.11.9)
File Edit Format Run Options Window Help
import unittest

def analyze_additive(scores):

    for s in scores:
        if s < 0 or s > 100:
            return "Invalid Score"

    avg = sum(scores) / len(scores)

    if avg < 30:
        return "Green Safe"
    elif avg < 60:
        return "Yellow Warning"
    else:
        return "Red Alert"

if __name__ == "__main__":

    print("AI Based Food Analyzer")
    print("Enter additive harm scores (0-100) separated by space:")

    user_input = input()

    scores = list(map(int, user_input.split()))

    result = analyze_additive(scores)

    print("Analysis Result:", result)

class TestFoodAnalyzer(unittest.TestCase):

    def test_invalid(self):
        self.assertEqual(analyze_additive([120, 80]), "Invalid Score")

    def test_green_safe(self):
        self.assertEqual(analyze_additive([10, 20]), "Green Safe")

    def test_yellow_warning(self):
        self.assertEqual(analyze_additive([40, 50]), "Yellow Warning")

    def test_red_alert(self):
        self.assertEqual(analyze_additive([80, 90]), "Red Alert")

    def test_multiple_inputs(self):

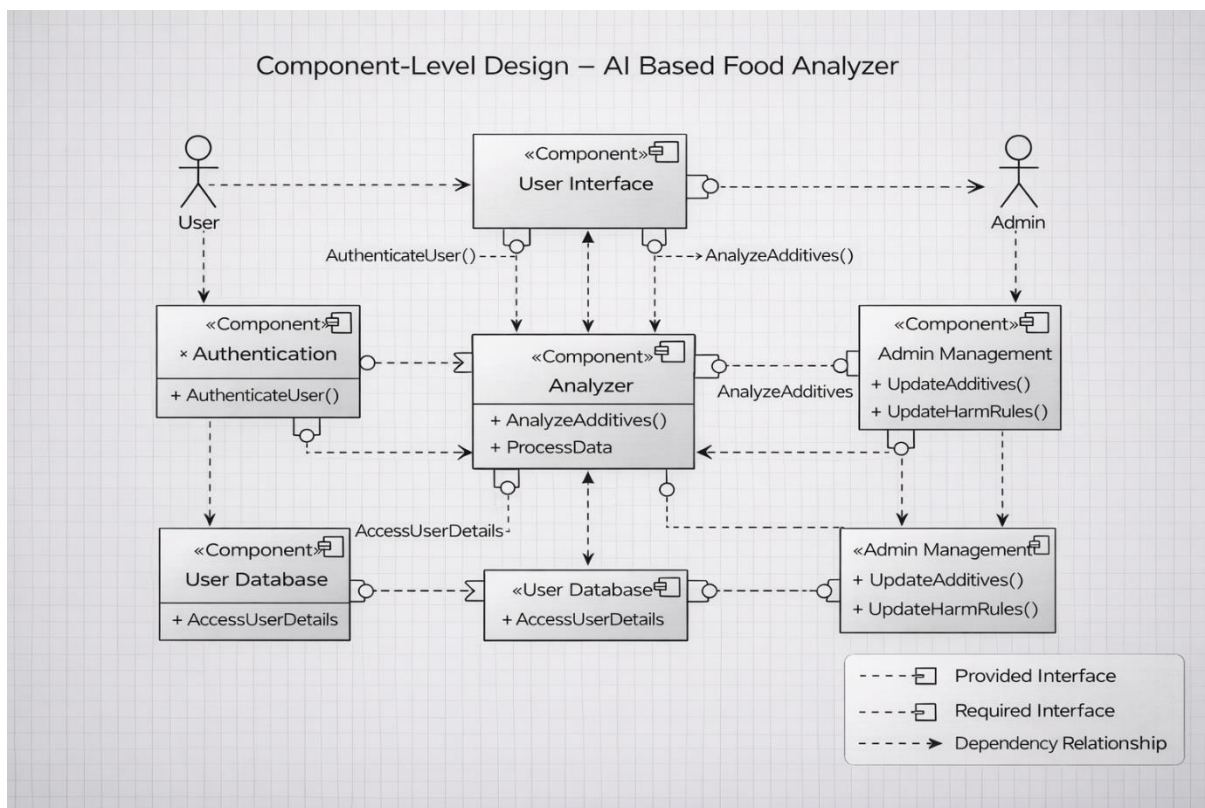
        - - -

    def test_multiple_inputs(self):
        self.assertEqual(analyze_additive([70, 80, 75]), "Red Alert")|
```

Output:

```
def test_multiple_inputs(self):
    self.assertEqual(analyze_additive([70, 80, 75]), "Red Alert")
```

Design Architecture of System:



1. User:

The User is the primary person interacting with the system. The user accesses the application to check the safety level of additives present in packaged food products.

The user can:

- Log into the system
- Enter food additive information
- View the analysis results generated by the system

All user requests are first handled through the User Interface.

2. User Interface Component:

The User Interface (UI) acts as a bridge between the user and the internal components of the system.

Its responsibilities include:

- Accepting additive information from the user
- Sending the data to the Analyzer component for processing
- Displaying the final analysis results to the user

The main operations related to this component are:

- InputAdditives()
- DisplayResult()

3. Authentication Component:

The Authentication component verifies the identity of the user before granting access to the system.

Its key function is:

- AuthenticateUser()

When a user attempts to log in, this component checks the credentials by accessing the User Database. If the information is correct, the system allows the user to continue using the application.

4. Analyzer Component:

The Analyzer is the central processing component of the system. It performs the main analysis tasks.

Its responsibilities include:

- Receiving additive data from the User Interface
- Searching the Additive Database for related information
- Applying predefined rules to determine the safety level of additives

The operations involved are:

- AnalyzeAdditives()
- ProcessData()

After processing the information, the Analyzer sends the results back to the User Interface.

5. User Database:

The User Database stores all information related to users of the system.

This database contains:

- User login credentials
- User profile information

The Authentication component accesses this database to validate user details during login.

The main operation used here is:

- AccessUserDetails()

6. Additive Database:

The Additive Database holds detailed data about different food additives.

It includes information such as:

- Additive codes or names
- Safety classification or harm levels

The Analyzer retrieves this information during the analysis process in order to evaluate the additives entered by the user.

7. Admin:

The Admin manages and maintains the system. The administrator ensures that the information stored in the system remains updated and accurate.

Admin responsibilities include:

- Updating additive records
- Managing system rules for harm classification
- Monitoring system operations

The Admin interacts with the Admin Management component.

8. Admin Management Component:

The Admin Management component provides administrative control over the system.

Its main functions are:

- UpdateAdditives()
- UpdateHarmRules()

Using this component, the admin can update the additive database and modify the rules used by the Analyzer.

9. Relationships Between Components:

The connections shown in the diagram indicate how components depend on each other to complete different tasks.

For example:

- The User Interface depends on the Analyzer to process additive information.
- The Analyzer depends on the Additive Database to retrieve additive data.
- The Authentication component relies on the User Database to verify login credentials.

These dependencies allow different modules of the system to work together efficiently.

Function-Based Metrics :

Food Adulteration Detection System:

Step 1: Identification of Functional Components:

To measure the system size, we first identify different types of user interactions and data handling elements.

External Inputs:

These are actions where users provide data to the system.

- User registration and login
- Entering daily health details (sleep, exercise, diet)
- Editing user profile
- Admin managing users

Total External Inputs = 4

External Outputs:

These represent processed information given to users.

- Health summary reports
- Dashboard display
- Alerts and notifications

Total External Outputs = 3

External Inquiries:

These are requests where users retrieve data without modifying it.

- Viewing health history
- Checking progress trends

Total External Inquiries = 2

Internal Logical Files:

These are data files maintained within the system.

- User information storage
- Health activity records
- System logs

Total Internal Logical Files = 3

External Interface Files:

These are data sources from outside the system.

- Data from fitness devices
- Third-party health applications

Total External Interface Files = 2

Step 2: Calculation of Unadjusted Function Points:

Using standard weights, we calculate the total:

Component	Count	Weight	Result
EI	4	4	16
EO	3	5	15
EQ	2	4	8
ILF	3	10	30
EIF	2	7	14

Total UFP = 83

Step 3: Complexity Adjustment Factor:

To reflect system complexity, we use:

$$\text{CAF} = 0.65 + (0.01 \times \Sigma Fi)$$

Assuming average complexity for all factors:

$$\Sigma Fi = 14 \times 3 = 42$$

$$\text{CAF} = 0.65 + (0.01 \times 42)$$

$$\text{CAF} = 0.65 + 0.42 = 1.07$$

Step 4: Final Function Point:

$$\text{FP} = \text{UFP} \times \text{CAF}$$

$$\text{FP} = 83 \times 1.07 = 88.81 \approx 89$$

Final Result:

The system size is approximately 89 Function Points.

Interpretation

- This indicates a moderately complex system
- The presence of multiple modules like analysis, reporting, and notifications increases functionality
- The system is suitable for institutional or multi-user environments

Effort Estimation:

If:

- 1 Function Point $\approx$ 10 hours

Then:

$$89 \times 10 = 890 \text{ hours}$$

If cost = ₹1000/hour:

Estimated Cost = ₹8,90,000

Conclusion:

The Health Routine Analysis System was evaluated using Function Point Analysis. After identifying all functional components and adjusting for system complexity, the final size was estimated to be 89 Function Points, indicating a system of moderate scale and functionality.

COCOMO Model:

Basic COCOMO Model:

Food Adulteration Detection System:

1. Introduction:

The COCOMO (Constructive Cost Model) is used to estimate the development effort and time required for a software project based on its size. The size is measured in KLOC (Kilo Lines of Code).

2. Given Information:

- Project Size = 50 KLOC (*assumed for this web-based system*)

Formulas used:

$$E = A \times (KLOC)^B$$

$$T = C \times (E)^D$$

Where:

- E = Effort (Person-Months)
- T = Development Time (Months)

3. Mode Constants:

Mode	A	B	C	D
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

4. Calculations:

(a) Organic Mode

Used for small and less complex systems

$$E = 2.4 \times (50)^{1.05}$$

$$E \approx 2.4 \times 60.41 = 144.98 \text{ PM}$$

$$T = 2.5 \times (144.98)^{0.38}$$

$$T \approx 2.5 \times 6.64 = 16.6 \text{ Months}$$

(b) Semi-Detached Mode

Used for medium-level projects (most suitable here)

$$E = 3.0 \times (50)^{1.12}$$

$$E \approx 3.0 \times 73.94 = 221.82 \text{ PM}$$

$$T = 2.5 \times (221.82)^{0.35}$$

$$T \approx 2.5 \times 6.67 = 16.67 \text{ Months}$$

(c) Embedded Mode

Used for highly complex systems

$$E = 3.6 \times (50)^{1.20}$$

$$E \approx 3.6 \times 109.33 = 393.59 \text{ PM}$$

$$T = 2.5 \times (393.59)^{0.32}$$

$$T \approx 2.5 \times 6.52 = 16.3 \text{ Months}$$

5. Final Summary:

Mode	Effort (PM)	Time (Months)
Organic	144.98	16.6
Semi-Detached	221.82	16.67
Embedded	393.59	16.3

6. Conclusion:

Based on the nature of the Health Routine Analysis System, which includes multiple modules such as data entry, analysis, reporting, and notifications, it falls under the Semi-Detached category.

Hence, the estimated development effort is 221.82 person-months, and the expected completion time is around 16.67 months.

Intermediate COCOMO Model

Food Adulteration Detection System:

1. Introduction

The Intermediate COCOMO model improves the basic model by including additional factors called cost drivers. These drivers consider aspects such as system reliability, complexity, team experience, and tools used.

These factors are combined into a single value called the Effort Adjustment Factor (EAF).

2. Given Data:

- Project Size = 50 KLOC (*assumed*)

Formulas:

$$E = A \times (KLOC)^B \times EAF$$

$$T = C \times (E)^D$$

$$\text{People Required} = \frac{E}{T}$$

3. Cost Drivers (EAF)

The Intermediate model considers 15 cost drivers grouped into:

(i) Product Attributes

- Software reliability
- Database size
- Product complexity

(ii) Hardware Attributes

- Execution time constraints
- Memory limitations
- Environment volatility
- Turnaround time

(iii) Personnel Attributes

- Analyst capability
- Programmer capability
- Application experience
- Platform experience
- Programming language experience

(iv) Project Attributes

- Use of software tools
- Development methods
- Schedule constraints

4. Assumption for EAF

For simplicity, assume all cost drivers are Nominal (1.0)

Therefore,

$$EAF = 1.0$$

5. Constants (Semi-Detached Mode – suitable for this project)

A	B	C	D
3.0	1.12	2.5	0.35

6. Calculations

Step 1: Effort

$$E = 3.0 \times (50)^{1.12} \times 1.0$$

$$E = 3.0 \times 73.94 = 221.82 \text{ Person-Months}$$

Step 2: Development Time

$$T = 2.5 \times (221.82)^{0.35}$$

$$T = 2.5 \times 6.67 = 16.67 \text{ Months}$$

Step 3: Number of Persons Required

$$\text{Persons} = \frac{221.82}{16.67} \approx 13$$

7. Final Result

- Effort = 221.82 Person-Months
- Development Time = 16.67 Months
- Team Size $\approx$ 13 Members

8. Conclusion

The Intermediate COCOMO model provides a more realistic estimate by considering system and team-related factors. Since the Health Routine Analysis System has moderate complexity and multiple modules, it fits best under the Semi-Detached mode.

The estimated effort is 221.82 person-months, with a development time of 16.67 months, requiring approximately 13 developers.